

TECH

JAVASCRIPT FUNDAMENTALS

EVENT LOOP / ASYNC

CLOSURES / SCOPE / MEMORY

PROTOTYPES / OBJECTS / CLASSES

EVENT LOOP / ASYNC

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК learn.javascript.ru КЭШ В `../learn-js-cache`

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 ЛЁГКИЙ	БАЗА EVENT LOOP И В ASYNC Q1 · Q19 · Q20	SETTIMEOUT	ОШИБК	Разогрев – три коротких темы, чтобы по имать общий каркас. Без этого дальш е будет тяжело.
ДЕНЬ 2 СРЕДНИЙ	МИКРО vs МАКРО UI Q2 · Q4 · Q5	PAINT	БЛОКИРОВКА	Дальше нюансы планирования: что попада ет в какую очередь, когда успевает рен дер и почему UI может зависнуть.
ДЕНЬ 3 ТЯЖЁЛЫЙ	ПАЗЛ ПОРЯДКА Q3	setTimeout / Promise.t hen / queueMicrotask / rAF		Один очень сложный вопрос – основной к оронный. Прогнать руками 5-10 пазлов с консолью.
ДЕНЬ 4 СРЕДНИЙ	PROMISE API Q6 · Q11 · Q12	AWAIT В ЦИКЛЕ	PARALL EL vs SEQUENTIAL	Базовый промис-инструментарий. Тут соб ируются три тесно связанных вопроса.
ДЕНЬ 5 ТЯЖЁЛЫЙ	RACE CONDITION Q9 · Q10	STALE RESPONSE В REA CT		Связка из двух – теория и практика на одну боль. На неё спрашивают почти все гда, понимание должно быть железное.
ДЕНЬ 6 ТЯЖЁЛЫЙ	ПРАКТИКА Q7 · Q8 · Q13	RETRY + BACKOFF CONCURRENCY	ABORT F	Кодинг-вопросы. Желательно руками проп исать retry и pool с лимитом параллели зма.
ДЕНЬ 7 ЛЁГКИЙ	DEBOUNCE / THROTTLE Q14 · Q15 · Q16 · Q17 · Q18	rAF / rIC	ST	Закрепление и тайминговые тонкости. На этот день не оставлять новую теорию – повторение и пазлы.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ЛЁГКИЙ 3 ВОПРОСА

БАЗА EVENT LOOP

SETTIMEOUT

ОШИБКИ В ASYNC

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Разогрев — три коротких темы, чтобы поймать общий каркас. Без этого дальше будет тяжело.

ВОПРОСЫ ДНЯ

Q01 Что такое event loop?

Q19 Почему `setTimeout(fn, 0)` не выполняется сразу?

Q20 Что произойдёт при ошибке внутри async-функции без try/catch?

Что такое event loop?

ОПИСАНИЕ

Event loop – это механизм, который позволяет однопоточному JS обрабатывать асинхронные операции, не блокируя поток исполнения. Технически это бесконечный цикл, организованный самой средой выполнения (браузером или Node.js), который связывает три сущности: стек вызовов, очереди задач и собственно код движка. Сам JS-движок (V8, JavaScriptCore, SpiderMonkey) умеет только одно – исполнять синхронные инструкции, забирая кадры из стека. Всё, что выглядит как «асинхронное», на самом деле выполняется не движком, а средой вокруг него – Web API в браузере, libuv в Node – и возвращается обратно как очередная задача.

КАК РАБОТАЕТ

01 JS-код выполняется в одном потоке через стек вызовов (call stack). Вход в функцию – push нового кадра; выход – pop. Когда стек пуст, движок ничего не делает и ждёт, пока ему дадут следующую задачу.

02 Стек глубже определённого порога – это RangeError 'Maximum call stack size exceeded'. Лимит зависит от движка и платформы, но в порядке десятков тысяч кадров для V8 в Node.

03 Долгие операции (таймеры, сеть, чтение файла, обработчики событий, обращения к индексированному хранилищу) сам JS-движок не делает. Их выполняет среда: в браузере это набор Web API, в Node – libuv с пулом потоков под I/O.

04 Когда среда заканчивает работу, она кладёт результат-колбэк в одну из очередей. Очередей две: macrotask queue и microtask queue.

05 Алгоритм event loop в упрощении: 1) если стек не пуст – ничего не делать, дождаться, пока опустеет; 2) если есть микрозадачи – выполнить их все подряд, не прерываясь; 3) только потом взять одну макрозадачу из очереди и положить её колбэк в стек.

06 В браузере между макрозадачами также вклинивается рендер-фаза: пересчёт стилей, layout, paint, composite. Поэтому длинная макрозадача = пропущенные кадры.

07 В Node event loop делится на фазы: timers, pending callbacks, idle/prepare, poll, check, close callbacks. process.nextTick и микрозадачи проигрываются между фазами и имеют приоритет выше «обычных» макротасков.

08 В Web Worker и в дочерних потоках Node.js есть собственный event loop с собственным стеком – поэтому Worker не блокирует главный поток UI.

ГДЕ ВСТРЕЧАЕТСЯ

01 Любая асинхронность в JS прошла через event loop: setTimeout, fetch, обработчики кликов, чтение файла в Node – без исключений.

02 Понимание нужно при дебаге странного порядка вывода: «почему console.log идёт ДО setTimeout, хотя написан после».

03 Профилирование: длинные тики event loop – это и есть janky UI. В Chrome DevTools во вкладке Performance видны блоки макрозадач и зазоры между ними, в которые вписывается рендер.

04 Web Worker – единственный штатный способ настоящей параллельности в браузерном JS. Это отдельный event loop, общающийся с главным через postMessage.

ПОДВОДНЫЕ КАМНИ

01 JS однопоточный – но среда вокруг (браузер, Node) многопоточная. Это часто путают: «JS не может в параллель» относится только к коду, не к работе платформы.

02 «Однопоточный» не значит «медленный» – пока среда работает (делает запрос, читает файл), JS-поток свободен и может обрабатывать другие задачи.

03 Один тяжёлый синхронный кусок блокирует ВСЁ: и таймеры, и события, и рендер. Event loop не сделает ничего, пока стек не освободится.

04 В Node setImmediate ставит макрозадачу в фазу check, а setTimeout(fn, 0) – в фазу timers; поэтому их относительный порядок может различаться от запуска к запуску, особенно вне I/O-колбэка.

КОД

// Минимальный пример порядка

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// 1, 4, 3, 2
// 1 и 4 – синхронный код
// 3 – микрозадача (then), идёт сразу после синхронного
// 2 – макрозадача (setTimeout), после микрозадач
```

// Стек переполнен

```
function rec() { rec(); }
rec();
// RangeError: Maximum call stack size exceeded
```

// Долгий синхронный блок убивает всё

```
setTimeout(() => console.log('таймер'), 100);
const start = Date.now();
while (Date.now() - start < 1000) {} // блокируем 1 сек
console.log('конец синхронного');
// конец синхронного
// таймер ← пришёл с задержкой ~1с, а не 100мс
```

МОГУТ ДОСПРОСИТЬ

01 Почему JS однопоточный – это ограничение спецификации или движка?

02 Что блокирует event loop?

03 Один ли event loop на вкладку браузера? (один на агента; Web Worker имеет собственный)

04 Чем event loop в Node отличается от event loop в браузере?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/event-loop>

ВОПРОС Q19

Почему `setTimeout(fn, 0)` не выполняется сразу?

ОПИСАНИЕ

`setTimeout` не запускает функцию через ровно ноль миллисекунд. Он только ставит её в очередь макрозадач, и она будет выполнена тогда, когда `event loop` до неё дойдёт. Это значит – после всего синхронного кода и после полного дренажа очереди микрозадач (в том числе той, что появится по ходу). Плюс у браузера есть минимальная задержка для вложенных таймеров – обычно 4 мс по спецификации HTML, чтобы рекурсивные `setTimeout` не съедали CPU. Поэтому даже в идеальном случае `setTimeout(fn, 0)` вызывает `fn` заметно позже, чем «прямо сейчас».

КАК РАБОТАЕТ

- 01 `setTimeout(fn, ms)` = «положи `fn` в макроочередь не раньше, чем через `ms` миллисекунд». Это верхнее ограничение со стороны клиентского кода – нижнее ставит сама среда.
- 02 До запуска `fn` должно произойти три условия: 1) истёк указанный таймер; 2) опустел `call stack`; 3) опустела очередь микрозадач (а она может пополняться по ходу).
- 03 HTML-спецификация требует: если `setTimeout` вложен в другой `setTimeout` уровня глубже 5, минимальная задержка – 4 мс. Поэтому `setTimeout(fn, 0)` внутри `setInterval`-цикла = 4 мс.
- 04 В неактивной вкладке таймеры и `rAF` урезаются ещё сильнее, обычно до 1 раза в секунду – браузер экономит CPU и батарею.
- 05 В Node поведение чуть другое: минимальная задержка близка к 1 мс, и `setImmediate` (фаза `check`) обычно срабатывает раньше `setTimeout(fn, 0)` внутри I/O-колбэка.
- 06 Если нужен «следующий тик прямо сейчас» – есть варианты: `queueMicrotask` (микро, в этом же тике), `MessageChannel` (макро без 4 мс клампа, через `postMessage`), `scheduler.postTask` (в современных браузерах).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 «Отложить выполнение на следующий тик» – частый паттерн перед ленивой загрузкой/инициализацией, чтобы не блокировать рендер.
- 02 Разбить тяжёлый цикл на чанки: каждый шаг – через `setTimeout(0)`, чтобы между ними успевал рендер и пользовательские события.
- 03 «Дождаться окончания текущего синхронного блока» – иногда так оборачивают `reflow`-зависимый код: сначала меняем DOM, затем через `setTimeout(0)` читаем геометрию.
- 04 Полифиллы для отсутствующего `queueMicrotask` раньше делали через `Promise.resolve().then` или через `MessageChannel` – фактически имитировали «следующий тик».

ПОДВОДНЫЕ КАМНИ

- 01 0 в `setTimeout` не значит «сразу» – на практике это «не раньше 4 мс» в большинстве браузеров для вложенного таймера и обычно несколько миллисекунд для верхнеуровневого.

02 Если в текущем тике запланировано много `.then`-ов – `setTimeout(0)` ждёт, пока они все отработают. На большой цепочке это может быть значительно дольше 4 мс.

03 Полагаться на точность `setTimeout` для синхронизации – нельзя. Для анимаций с привязкой к кадру – `requestAnimationFrame`.

04 `setTimeout` с большим `ms` ($>2^{31}$) ведёт себя как 0 из-за переполнения 32-битного `int` – такой код часто пишут случайно при подсчёте «отложить на 30 дней».

код

// Сначала микро, потом `setTimeout`

```
setTimeout(() => console.log('macro'), 0);
Promise.resolve().then(() => console.log('micro'));
// micro
// macro
```

// Задержка 0 при множестве микрозадач

```
setTimeout(() => console.log('timeout 0'), 0);
let p = Promise.resolve();
for (let i = 0; i < 1000; i++) {
  p = p.then(() => {});
}
p.then(() => console.log('1000 then'));
// 1000 then
// timeout 0 ← ждал, пока цепочка отработает
```

// Чанки тяжёлой работы

```
function chunk(items, work) {
  if (!items.length) return;
  const slice = items.splice(0, 100);
  slice.forEach(work);
  // отдаём управление event loop'у – рендер успеет
  setTimeout(() => chunk(items, work), 0);
}
```

МОГУТ ДОСПРОСИТЬ

01 Чем `setTimeout(0)` отличается от `queueMicrotask(fn)` на практике?

02 Как добиться «следующего тика» без задержки 4 мс?

03 Откуда в спецификации взялась минимальная задержка 4 мс?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/settimeout-setinterval>

ВОПРОС Q20

Что произойдёт при ошибке внутри async-функции без try/catch?

ОПИСАНИЕ

Async-функция всегда возвращает промис — это её главный контракт, независимо от того, что внутри. Любая ошибка внутри (выкинутый `throw` или `rejected`-промис из `await`) переводит возвращаемый промис в состояние `rejected` с этой ошибкой в качестве причины. Дальнейшая судьба ошибки зависит от вызывающей стороны: если на промис повесили `.catch` или `await` обернули в `try/catch` — поймаем там; если никто не поймал — это становится `unhandled rejection`, глобальное событие, на которое можно подписаться. Внутри функции после непоиманной ошибки выполнение прекращается — так же, как в обычной синхронной функции после непоиманного `throw`.

КАК РАБОТАЕТ

- 01 `async function f() { throw e }` полностью эквивалентно `function f() { return Promise.reject(e) }` — это правило вшито в семантику ключевого слова `async`.
- 02 `await` на `rejected`-промисе ведёт себя как `throw`: исключение выкидывается в текущей точке выполнения. Без `try/catch` оно всплывает наружу `async`-функции и переводит её собственный промис в `rejected`.
- 03 Если на возвращённый промис никто не повесил `.catch` / `.then` (второй аргумент) / `await` в `try/catch` — он считается `unhandled`. Среда регистрирует это и выпускает событие: `window.unhandledrejection` в браузере, `process.on('unhandledRejection', ...)` в Node.
- 04 В Node поведение менялось: до 15.x — `warning` в консоль; с 15.x по умолчанию — это причина для падения процесса (`--unhandled-rejections=throw`). Это можно изменить флагом.
- 05 В браузере `unhandledrejection` не валит вкладку, но попадает в DevTools и обычно в Sentry/Rollbar/любой `error-tracker`.
- 06 Если ошибку поймали позже (привязали `.catch` к уже `rejected`-промису асинхронно) — событие `unhandledrejection` отменяется через `rejectionhandled`. Это позволяет «отложено» подписываться.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой `async`-эндпоинт обычно оборачивают `try/catch` внутри или `.catch` снаружи — иначе сломанная сеть положит UI или процесс.
- 02 Глобальный `unhandledrejection` — хороший крюк для логирования и алертов, но не замена локальной обработке.
- 03 В Express/Koa middleware — обязательное оборачивание `async`-роутов в `catch` (или библиотечный `wrapper` типа `express-async-errors`). Иначе ошибка падёт мимо обработчика ошибок Express.
- 04 В тестах `async`-функций — `await + expect().rejects.toThrow` или `try/catch` с проверкой ошибки. Если просто вызвать без `await` — тест может зелёным завершиться, а ошибка прилетит позже.

ПОДВОДНЫЕ КАМНИ

- 01 `throw` внутри `async`-функции виден снаружи только через `.catch` или `try/catch` вокруг `await` – другого способа нет. Просто `synchronous-style try { f() } catch` не поймает ошибку.
- 02 Если `async`-функцию вызвали и НЕ сохранили промис – никто его не «поймает», и ошибка превратится в `unhandledrejection`. Типичный кейс – `fire-and-forget` в обработчике клика.
- 03 `Array.forEach` с `async`-колбэком игнорирует возвращённые промисы – значит, ошибки внутри будут `unhandled rejection`, а сам `forEach` вернёт управление до завершения работы колбэков.
- 04 В обработчиках событий (`addEventListener`) `async`-колбэк тоже теряет управление: само событие не дождётся, и ошибки превратятся в `unhandled`, если внутри нет `try/catch`.
-

КОД

// Стандартный шаблон ловли ошибки

```
async function load() {
  const res = await fetch('/api');
  if (!res.ok) throw new Error('bad status: ' + res.status);
  return res.json();
}

// снаружи
load()
  .then(data => render(data))
  .catch(err => report(err));

// или внутри другого async
try {
  const data = await load();
} catch (err) {
  report(err);
}
```

// Глобальный перехват

```
// браузер
window.addEventListener('unhandledrejection', e => {
  e.preventDefault(); // не пускаем в DevTools
  report(e.reason);    // отправляем в Sentry
});

// Node
process.on('unhandledRejection', err => {
  report(err);
  // важно: после этого Node может убить процесс,
  // зависит от --unhandled-rejections
});
```

// Fire-and-forget – типичный источник `unhandled`

```
button.addEventListener('click', () => {
  doAsyncWork(); // промис никем не подхвачен
});
```

```
// безопаснее:  
button.addEventListener('click', () => {  
  doAsyncWork().catch(report);  
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Поведение Node при unhandled rejection – что меняется по версиям?
- 02 Что вернёт async function f() { throw 1 }, если её не await-ить?
- 03 Что произойдёт, если внутри async-функции выкинуть и тут же поймать ошибку, а потом снова выкинуть?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/async-await>

МИКРО vs МАКРО**PAINT****БЛОКИРОВКА UI****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Дальше нюансы планирования: что попадает в какую очередь, когда успевает рендер и почему UI может зависнуть.

ВОПРОСЫ ДНЯ

Q02 Чем task отличается от microtask?

Q04 Когда браузер получает шанс на paint?

Q05 Почему бесконечная microtask queue может заблокировать UI?

ВОПРОС Q02

Чем task отличается от microtask?

ОПИСАНИЕ

Task (макрозадача) и microtask – это две разные очереди задач в event loop, у которых разный приоритет и разные источники. Главное отличие – в порядке: после каждой макрозадачи движок ПОЛНОСТЬЮ опустошает очередь микрозадач, и только потом берёт следующую макрозадачу. Микрозадачи также опустошаются между выполнением скрипта и любой следующей фазой event loop. Источники тоже разные: setTimeout и обработчики событий порождают макрозадачи; .then у промисов и queueMicrotask порождают микрозадачи. Из этого вытекает важное следствие: Promise.then всегда обрабатывается раньше setTimeout(fn, 0), даже если оба запланированы в один момент.

КАК РАБОТАЕТ

- 01 Очереди две: macrotask queue (в HTML-спеке – task queue) и microtask queue (в HTML-спеке – microtask queue / job queue).
- 02 Что попадает в macrotask: setTimeout, setInterval, setImmediate (Node), обработчики DOM-событий, postMessage, MessageChannel, I/O-колбэки в Node.
- 03 Что попадает в microtask: колбэки .then / .catch / .finally, продолжения после await, queueMicrotask, MutationObserver. В Node – также process.nextTick, причём с приоритетом ВЫШЕ обычных микрозадач.
- 04 Алгоритм одного «оборота» event loop: 1) взять одну макрозадачу из очереди; 2) выполнить её до конца; 3) слить ВСЕ микрозадачи, включая те, что добавились по ходу; 4) (в браузере) при необходимости провести рендер-фазу; 5) повторить.
- 05 Микрозадачи дренируются «без передышек»: если внутри одной микрозадачи поставить ещё одну – она войдёт в этот же дренаж, а не в следующий.
- 06 В браузере между макрозадачами событиями ОДНОГО типа (например, несколько кликов) тоже могут вклиниваться микрозадачи – каждый клик обрабатывается как отдельная макрозадача.
- 07 В Node микрозадачи дренируются после каждой фазы event loop (timers, poll, check, ...), а не только после макрозадач.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая работа с промисами и async/await проходит через микроочередь – это основной кирпич современной асинхронности.
- 02 queueMicrotask(fn) – когда нужно отложить вызов «на конец текущего тика, но раньше любого setTimeout / события». Полезно для batched-обновлений в библиотеках реактивности.
- 03 MutationObserver – это микрозадача: реагирует на изменения DOM до следующего рендера, что позволяет отлавливать каскадные правки без миганий.
- 04 Внутри библиотек state-management часто используют queueMicrotask или Promise.resolve().then, чтобы откладывать уведомление подписчиков до конца текущей синхронной операции (пример – реактивность Vue, авто-batching React 18).

ПОДВОДНЫЕ КАМНИ

- 01 Длинная цепочка `.then` выполнится ДО следующего `setTimeout`, даже если в `setTimeout` стоит 0. Промисы – это микрозадачи и имеют приоритет.
- 02 Бесконечно растущая микроочередь блокирует UI: рендер не может вклиниться, пока не опустеет (см. Q5).
- 03 `queueMicrotask` и `Promise.resolve().then(fn)` – практически эквивалентны и идут в одну общую микроочередь, сохраняя порядок постановки. Но `queueMicrotask` дешевле: не создаёт новый промис.
- 04 В Node `process.nextTick` – НЕ микрозадача в строгом смысле, у неё своя ещё более приоритетная очередь. Рекурсивный `nextTick` душит все микрозадачи и фазы event loop.
- 05 Между микрозадачей и rAF разный момент исполнения: микро опустошаются раньше rAF, поэтому если в `.then` «попросить кадр», сначала отработает текущая микро-очередь, а потом – rAF.

КОД

// Микро всегда раньше следующей макро

```
setTimeout(() => console.log('macro'), 0);
Promise.resolve()
  .then(() => console.log('micro 1'))
  .then(() => console.log('micro 2'));
// micro 1
// micro 2
// macro
```

// Микро, добавленная по ходу – в тот же дренаж

```
Promise.resolve().then(() => {
  console.log('A');
  Promise.resolve().then(() => console.log('B'));
});
Promise.resolve().then(() => console.log('C'));
// A
// C
// B ← добавлена внутри A, попала в текущий дренаж
```

// Node – `process.nextTick` опаснее

```
setImmediate(() => console.log('immediate'));
Promise.resolve().then(() => console.log('promise'));
process.nextTick(() => console.log('nextTick'));
// nextTick
// promise
// immediate
```

// `queueMicrotask`

```
console.log('start');
queueMicrotask(() => console.log('micro'));
console.log('end');
// start
// end
// micro
```

МОГУТ ДОСПРОСИТЬ

- 01 `queueMicrotask` vs `Promise.resolve().then(fn)` – разница?
- 02 Почему долгая микроцепочка опасна для UI?
- 03 Что в Node быстрее – `process.nextTick` или `queueMicrotask`?
- 04 Какие наблюдатели DOM кладут микрозадачи?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/microtask-queue>

Когда браузер получает шанс на paint?

ОПИСАНИЕ

Paint (отрисовка пикселей на экран) — это часть рендер-фазы, которая в спецификации HTML вклинивается между макрозадачами. Но не после каждой: браузер сам решает, когда вставлять рендер-фазу, ориентируясь на частоту обновления экрана (обычно 60 fps, на современных дисплеях — 120). До paint в той же фазе движок выполняет несколько шагов: колбэки `requestAnimationFrame`, пересчёт стилей, `layout`, `paint`, `composite`. Микрозадачи всегда успевают опустошиться до начала рендер-фазы. Из этого следует ключевое: ни одна цепочка `.then`-ов не «прервётся» на рендер — пока она не доработает, paint не случится.

КАК РАБОТАЕТ

01 Браузер целится в кадры с интервалом ≈ 16.7 мс (60 fps). На 120-герцовых дисплеях — 8.3 мс. Если кадр не уложится в этот бюджет — он будет пропущен и пользователь увидит «дёрганье».

02 Один кадр в упрощённом виде: текущая макрозадача → опустошить микроочередь → вызвать колбэки `requestAnimationFrame` → пересчитать стили → `layout` → `paint` → `composite`.

03 Микрозадачи опустошаются ДО рендера. Поэтому paint не может вклиниться между `.then`-ами одной цепочки, как бы долго она ни выполнялась.

04 rAF-колбэки — особая фаза перед paint. Они идеальное место для апдейта анимаций: то, что вы измените в rAF, попадёт в этот же кадр.

05 Тяжёлая макрозадача (долгий `for`, тяжёлый `JSON.parse`, синхронная распаковка) приводит к пропуску кадров — `visual lag`, не реагируют скролл и клики.

06 В скрытой вкладке таймеры замедляются, rAF не вызывается вообще — paint не происходит. После возвращения вкладки браузер обычно делает один «догоняющий» рендер.

ГДЕ ВСТРЕЧАЕТСЯ

01 При написании JS-анимаций — апдейт визуала через `requestAnimationFrame`, не через `setTimeout`.

02 Тяжёлые вычисления — выносить в чанки через `setTimeout`, `requestIdleCallback` или `Web Worker`, чтобы рендер успевал.

03 При оптимизации скролл-обработчика: внутри сам обработчик ставит флажок, апдейт UI — внутри rAF. Так избегаем `layout thrashing` и попадаем в один кадр.

04 Перед измерением геометрии (`offsetWidth`, `getBoundingClientRect`) после изменения DOM — иногда дешевле подождать rAF, чтобы браузер успел сделать `layout` без принудительного `reflow`.

ПОДВОДНЫЕ КАМНИ

01 Рендер не обязан быть после каждой макрозадачи — браузер пропустит рендер, если кадр уже свежий или вкладка скрыта.

02 Layout thrashing: read → write → read → write в одном тике вызывает многократный пересчёт лэйаута. Все чтения – вверх, все записи – вниз.

03 rAF-колбэк выполняется в середине кадра, не «перед всем». Поэтому очень тяжёлый rAF тоже может не успеть отрисовать кадр.

04 Длинная микроочередь, добавляемая в .then-цепочке, тоже блокирует рендер – это другая сторона той же монеты, что в Q5.

05 В background-вкладке rAF молчит – анимация замёрзнет до возврата.

КОД

// Кадр пропущен из-за тяжёлой задачи

```
setInterval(() => {
  const start = Date.now();
  while (Date.now() - start < 100) {} // 100мс блока
}, 200);
// каждые 200мс – 100мс CPU,
// fps падает до ~5
```

// Скролл через rAF

```
let pending = false;
window.addEventListener('scroll', () => {
  if (pending) return;
  pending = true;
  requestAnimationFrame(() => {
    updateUI();
    pending = false;
  });
});
```

// Layout thrashing – антипаттерн

```
// плохо: чередование read/write вызывает reflow на каждом шаге
for (const el of items) {
  const w = el.offsetWidth; // read
  el.style.width = (w + 10) + 'px'; // write
}

// хорошо: сначала все read, потом все write
const widths = items.map(el => el.offsetWidth);
items.forEach((el, i) => {
  el.style.width = (widths[i] + 10) + 'px';
});
```

МОГУТ ДОСПРОСИТЬ

01 В каком порядке вызываются rAF-колбэки относительно микрозадач?

02 Что такое layout thrashing и как он связан с paint?

03 Что произойдёт со встроенной видеоплеер-анимацией в фоновой вкладке?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/event-loop>

Почему бесконечная `microtask queue` может заблокировать UI?

ОПИСАНИЕ

Микроочередь дренируется ПОЛНОСТЬЮ перед каждым рендером и перед следующей макрозадачей. Это правило вшито в спецификацию: пока очередь не пуста, браузер не идёт в `paint`, не берёт макрозадачу, не реагирует на клики. Если внутри микрозадачи добавлять новую микрозадачу – а та ещё одну – очередь никогда не опустеет. Получается ситуация, в которой JS-движок «работает», а UI стоит: не реагирует на ввод, не отрисовывается, таймеры не срабатывают. Снаружи это выглядит как зависший браузер. Поэтому рекурсивный `Promise.then`-цикл – это серьёзный антипаттерн, в отличие от обычного `while(true)`, о котором обычно сразу думают.

КАК РАБОТАЕТ

- 01 После каждой макрозадачи `event loop` сливает ВСЕ микрозадачи, включая те, что появились по ходу слива. Это правило нельзя обойти.
- 02 Если внутри `.then` снова добавлять `.then` – это тот же дренаж, никогда не заканчивающийся. JS-движок в полном порядке, он честно работает.
- 03 Получаем не `deadlock`, а `starvation`: код выполняется, но рендер, макро-задачи и события встают.
- 04 Лечение – переход на макрозадачу: `setTimeout(loop, 0)`, `MessageChannel.postMessage` или `scheduler.postTask`. Тогда между итерациями `event loop` успевает рендер и обработку событий.
- 05 В Node аналогичная проблема – `process.nextTick`, добавляющий ещё `nextTick`: эта очередь приоритетнее микрозадач и поэтому блокирует ещё больше.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Знание помогает дебажить ситуации «всё работает, но UI замёрз». В Performance-вкладке DevTools видно, что JS грузит CPU 100%, но фрейм-таймлайн пуст – это маркер именно микро-старвейшена.
- 02 В библиотеках реактивности и `auto-batching` очень осторожны с микро-очередью: накапливают до конца тика, потом обрабатывают.
- 03 Производственный код такую проблему обычно создаёт по ошибке: рекурсивный `then-loop` вместо итерации с `setTimeout`, или забытый `.then` на промисе, который сам себя резолвит цепочкой.

ПОДВОДНЫЕ КАМНИ

- 01 Длинная `.then`-цепочка ФИКСИРОВАННОЙ длины не блокирует UI бесконечно – она когда-то закончится. Опасна именно самовоспроизводящаяся очередь.
- 02 В отличие от `while(true)`, здесь не валится профайлер: код корректно отдаёт управление между микрозадачами, просто никогда не выходит из дренажа.
- 03 Похожая ловушка в Node – рекурсивный `process.nextTick`, ещё более приоритетный, чем микрозадачи. Можно заблокировать целые фазы `event loop`.

04 Переход на `setImmediate` в Node решает проблему, но делает цикл медленнее — между итерациями проходит целый «оборот» event loop.

КОД

// Вешаем UI

```
function loop() {
  Promise.resolve().then(loop);
}
loop();
// браузер встал: paint, события, setTimeout — ничего
```

// То же безопасно — на макрозадачах

```
function loop() {
  setTimeout(loop, 0); // браузер дышит между итерациями
}
loop();
```

// MessageChannel — макро без 4мс клампа

```
const ch = new MessageChannel();
function schedule(fn) {
  ch.port1.onmessage = fn;
  ch.port2.postMessage(null);
}
function loop() { doWork(); schedule(loop); }
loop();
```

МОГУТ ДОСПРОСИТЬ

- 01 А что с длинной `.then`-цепочкой без рекурсии — она заблокирует UI?
- 02 В чём разница между этой проблемой и обычным синхронным `while(true)`?
- 03 Почему `process.nextTick` в Node может быть опаснее микрозадачи?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/microtask-queue>

ПАЗЛ ПОРЯДКА `setTimeout` / `Promise.then` / `queueMicrotask`

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Один очень сложный вопрос — основной коронный. Прогнать руками 5-10 пазлов с консолью.

ВОПРОСЫ ДНЯ

Q03 В каком порядке выполнятся `setTimeout`, `Promise.then`, `queueMicrotask`, `requestAnimationFrame`?

ВОПРОС Q03

В каком порядке выполнятся `setTimeout`, `Promise.then`, `queueMicrotask`

ОПИСАНИЕ

Это классический вопрос на понимание event loop. Чтобы ответить, нужно держать в голове три уровня приоритета и одну отдельную фазу. Уровни приоритета: синхронный код самый главный, затем дренаж микрозадач, затем рендер-фаза (с rAF внутри неё), затем берётся следующая макрозадача – и весь цикл повторяется. Отдельная фаза – `requestAnimationFrame`: его колбэк не микро- и не классическая макрозадача, он живёт внутри рендер-фазы перед самым `paint`. Поэтому базовая модель ответа: sync → micro (then / `queueMicrotask`, в порядке постановки) → rAF → следующая макро (`setTimeout`) → опять micro и так далее.

КАК РАБОТАЕТ

- 01 Синхронный код выполняется первым, до любых очередей – он должен исчерпать стек, прежде чем event loop возьмёт что-то из очередей.
- 02 После каждой макрозадачи event loop опустошает очередь микрозадач: `.then`, `.catch`, `.finally`, `await`-продолжения, `queueMicrotask`, `MutationObserver`. Порядок – постановки в очередь.
- 03 `queueMicrotask` и `Promise.resolve().then(fn)` идут в одну общую микроочередь и сохраняют порядок добавления.
- 04 `requestAnimationFrame` – отдельная фаза «перед кадром», выполняется максимум один раз за рендер-фазу. Базовая модель: текущая макро → микро → rAF → стили → layout → paint.
- 05 `setTimeout` – обычная макрозадача. Если задержка 0 – попадает в макроочередь, но всё равно ждёт, пока освободятся стек и микроочередь.
- 06 `await` – это микрозадача в маске: всё, что после `await`, идёт как `.then`-продолжение. Пишите `async function f() { console.log('a'); await null; console.log('b'); }` – между 'a' и 'b' будет переход в микроочередь.
- 07 Микрозадача, добавленная во время дренажа, тоже выполняется в этом же дренаже. Поэтому в типичном пазле первая `.then` может добавить вторую, и обе выполнятся до `setTimeout`.
- 08 `MutationObserver` – микро. `ResizeObserver` и `IntersectionObserver` имеют свой порядок – обычно перед `requestAnimationFrame`, но это не классическая микрозадача.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Пазлы на собеседованиях – почти каждый разговор про event loop проверяют примерно таким куском кода. Это самый частый формат вопроса по async.
- 02 Дебаг неожиданного порядка вывода в реальных приложениях: консоль показывает не то, что ожидаешь – почти всегда виноват путанный порядок макро/микро.
- 03 Понимание помогает при оптимизации UI – что когда успеет и где ставить «отложенный» код.

ПОДВОДНЫЕ КАМНИ

- 01 Поведение rAF может слегка отличаться между браузерами и Node-like средами (в Node нет rAF, но есть аналоги в scheduling-API).
- 02 `await` — это всегда микрозадача, даже если ждёт уже разрешённый промис. `async function f() { await 1; }` всё равно делает переход в микроочередь.
- 03 В одной макрозадаче добавленная микро-задача попадает в этот же дренаж — это часто удивляет. Логика «следующая микро на следующий тик» неверна.
- 04 `scheduler.postTask` с приоритетом `user-blocking` ставит задачи в макроочередь, а с `background` — гораздо позже. Это новый API, в старых средах его нет.
- 05 В Node `process.nextTick` идёт ПЕРЕД микрозадачами — то есть ещё приоритетнее, чем `.then`.

КОД

// Классический пазл

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
queueMicrotask(() => console.log('D'));
requestAnimationFrame(() => console.log('E'));
console.log('F');
// A, F — синхронный код
// C, D — микрозадачи в порядке постановки
// E — rAF, фаза перед paint
// B — следующая макрозадача (setTimeout)
```

// Микро добавляется по ходу

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => {
  console.log(3);
  Promise.resolve().then(() => console.log(4));
});
console.log(5);
// 1, 5
// 3, 4 — добавленный then тоже в текущий дренаж
// 2
```

// `await` — микрозадача в маске

```
async function f() {
  console.log('a');
  await null;
  console.log('b');
}
f();
console.log('c');
// a — синхронно до await
// c — синхронный хвост
// b — продолжение функции, как .then
```

// Цепочка из нескольких `then` против `setTimeout`

```
setTimeout(() => console.log('timeout'), 0);
Promise.resolve()
```

```
.then(() => console.log('then 1'))  
.then(() => console.log('then 2'))  
.then(() => console.log('then 3'));  
// then 1, then 2, then 3, timeout
```

МОГУТ ДОСПРОСИТЬ

- 01 Что напечатает порядок, если внутри rAF добавить queueMicrotask?
- 02 Что добавляет MutationObserver – макро или микро?
- 03 В Node: где встанет process.nextTick относительно микрозадач?
- 04 Чем scheduler.postTask отличается от setTimeout по приоритету?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/event-loop>

PROMISE API**AWAIT В ЦИКЛЕ****PARALLEL vs SEQUENTIAL****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Базовый промис-инструментарий. Тут собираются три тесно связанных вопроса.

ВОПРОСЫ ДНЯ

Q06 Чем `Promise.all`, `Promise.allSettled`, `Promise.race`, `Promise.any` отличаются друг от друга?

Q11 Что происходит при `await` внутри цикла?

Q12 Когда лучше использовать `parallel execution`, а когда `sequential`?

ВОПРОС Q06

Чем Promise.all, Promise.allSettled, Promise.race, Promise.any отл

ОПИСАНИЕ

Все четыре – статические методы класса Promise, которые принимают итерабельный объект промисов и возвращают один общий промис. Разница в том, ЧТО они считают результатом и КАК реагируют на ошибки. Promise.all – самый строгий: ждёт успеха ВСЕХ, падает по первой ошибке. Promise.allSettled – самый толерантный: ждёт всех, никогда не падает, отдаёт массив с маркером «успех/ошибка» для каждого. Promise.race отдаёт результат первого ЗАВЕРШИВШЕГОСЯ, любого исхода. Promise.any отдаёт первый успешный, если все упали – AggregateError со списком причин. Эти четыре покрывают большинство сценариев параллельной координации промисов.

КАК РАБОТАЕТ

- 01 Promise.all([...]) – параллельный запуск + единый успех. Резолвится массивом значений в исходном порядке (не в порядке завершения). Падает по первой ошибке – остальные промисы не отменяются, просто их результаты больше никого не интересуют.
- 02 Promise.allSettled([...]) – собрать всё. Каждый элемент результата: {status: 'fulfilled', value: ...} либо {status: 'rejected', reason: ...}. Никогда не падает. Появился в ES2020.
- 03 Promise.race([...]) – первый завершившийся, любой исход. Если первый промис упал – общий промис тоже упадёт, даже если остальные были бы успешными.
- 04 Promise.any([...]) – первый успешный. Если все упали – AggregateError со списком причин в e.errors. Появился в ES2021.
- 05 Все методы работают в режиме «лениво НЕ отменяют»: даже после разрешения общего промиса остальные продолжают работать в фоне до своего естественного финала. Для отмены fetch – AbortController вручную.
- 06 Если передан пустой итератор: Promise.all([]) и Promise.allSettled([]) сразу резолвятся пустым массивом; Promise.race([]) – навсегда pending; Promise.any([]) – сразу AggregateError.
- 07 Все методы принимают любой итератор, не только массив: Promise.all(new Set([p1, p2])) тоже работает.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Promise.all – когда нужны ВСЕ результаты и любая ошибка означает провал операции. Например, страница из 3 endpoint-ов: без одного из них рендер не имеет смысла.
- 02 Promise.allSettled – дашборд из независимых виджетов: один из API упал, остальные показываем. Также полезно при batch-операциях: «попробовать удалить 50 файлов и сообщить, какие не удалось».
- 03 Promise.race – таймауты: race(actualWork, timeoutPromise). Также – «успей первый»: загрузить из кэша и из сети одновременно, взять что вернётся быстрее.
- 04 Promise.any – fallback-цепочки: попробовать список зеркал, взять первое живое. Удобно в realtime-системах с балансировкой.

05 В тестах часто `Promise.allSettled` – чтобы тест дождался всех промисов и не упал на первом, скрыв проблемы в остальных.

ПОДВОДНЫЕ КАМНИ

01 Ни один из методов не отменяет «лишние» промисы. Запросы продолжают идти, занимая ресурсы. Для отмены `fetch` – `AbortController` вручную (см. Q8).

02 `Promise.all` сохраняет порядок РЕЗУЛЬТАТОВ, а не порядок завершения. Если важно «кто первый» – это `race`.

03 `Promise.race` реагирует на первое событие, и неважно, успех или ошибка. Если первая ошибка пришла раньше первого успеха – упадёт весь `race`. Это часто причина flaky-тестов.

04 `Promise.any` – относительно новый (ES2021), в старых средах нужен полифилл (например, `core-js`).

05 `AggregateError` – отдельный класс ошибки. Чтобы достать оригинальные причины – `e.errors` (массив).

06 `Promise.all` со 1000 промисов – все 1000 запросов уйдут одновременно. На бэкенде это либо 429, либо забитый `connection pool`.

КОД

// all – все или никто

```
const [user, posts, comments] = await Promise.all([
  fetchUser(id),
  fetchPosts(id),
  fetchComments(id),
]);
// если хотя бы один await упал – попадаем в catch
// и не получаем НИЧЕГО, даже частичных результатов
```

// allSettled – частичный результат

```
const results = await Promise.allSettled(
  urls.map(u => fetch(u))
);
const ok      = results.filter(r => r.status === 'fulfilled');
const failed = results.filter(r => r.status === 'rejected');
ok.forEach(r => render(r.value));
failed.forEach(r => log(r.reason));
```

// race – таймаут

```
function timeout(ms) {
  return new Promise((_, reject) =>
    setTimeout(() => reject(new Error('timeout')), ms)
  );
}
try {
  const data = await Promise.race([
    fetch('/slow').then(r => r.json()),
    timeout(5000),
  ]);
} catch (e) {
  // либо ошибка fetch, либо timeout
}
```

// any – fallback

```
try {
  const first = await Promise.any([
    fetch('https://mirror1.example/'),
    fetch('https://mirror2.example/'),
    fetch('https://mirror3.example/'),
  ]);
  use(first);
} catch (e) {
  if (e instanceof AggregateError) {
    e.errors.forEach(log); // все упали
  }
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда выгоднее `allSettled`, чем `all`?
- 02 Чем `Promise.any` отличается от `race`?
- 03 Как реализовать «первый успешный за N секунд, иначе ошибка»?
- 04 Что произойдёт, если в `Promise.all` передать массив с уже разрешёнными промисами?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/promise-api>

Что происходит при `await` внутри цикла?

ОПИСАНИЕ

`await` ставит выполнение `async`-функции на паузу до разрешения промиса. Внутри цикла это значит, что итерации идут строго друг за другом – следующая не начнётся, пока текущая не дождалась своего `await`. Это важно понимать, потому что в большинстве реальных задач это не то, чего хотелось: задачи часто независимы, и последовательный `await` превращает пачку быстрых параллельных запросов в медленную ленту. Альтернатива – собрать все промисы и применить `Promise.all` (если задачи независимы) или сделать пул с лимитом (если их много и нужно щадить бэкенд).

КАК РАБОТАЕТ

- 01 `for / while + await` – последовательное выполнение, итерации одна за другой. Время растёт линейно: $N \text{ задач} \times \text{среднее время одной}$.
- 02 `for-of + await` читается чище, чем `for-i + await`, и тоже последовательный.
- 03 `forEach + async` внутри НЕ работает как ожидается: `forEach` игнорирует возвращаемый промис, итерации не ждут. Колбэк-функция `async` становится `fire-and-forget`.
- 04 Для параллели независимых задач: `arr.map(asyncFn) → Promise.all`. `map` создаёт массив промисов синхронно, все стартуют сразу.
- 05 Для лимитированной параллели – пул воркеров (см. Q13).
- 06 `for-await-of` – для `async`-итераторов: обходит `ReadableStream`, `async`-генератор или серию страниц с пагинацией. Семантика – тоже последовательная.
- 07 Под капотом `await` компилируется в `.then`-продолжение: функция приостанавливается, оставшийся хвост улетает в микроочередь, дожидается, потом возобновляется.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Последовательно – когда есть зависимость по данным или важен порядок: серия записей в одну сущность, миграция шагами, пагинация по `cursor` (где `cursor` для $N+1$ берётся из ответа N).
- 02 Параллельно – когда задачи независимы: загрузка нескольких ресурсов, запросы к разным API, обработка пачки изображений.
- 03 `for-await-of` – для перебора стрима или асинхронного генератора. Например, чтение чанков из `ReadableStream` при потоковом ответе сервера.
- 04 В скриптах CLI/build часто последовательно – чтобы логи были предсказуемые.

ПОДВОДНЫЕ КАМНИ

- 01 Главный антипаттерн – последовательный `await` там, где задачи независимы. Время растёт линейно от количества задач.
- 02 `forEach + async` – частая ошибка: код не дожидается итераций, а ошибки внутри становятся `unhandled rejection`.

03 В `Promise.all` все промисы стартуют одновременно – на бэкенд может прилететь `N` одновременных запросов. На больших `N` это проблема (см. Q13 про `concurrency`).

04 `await` в цикле в синхронном коллбэке (например, в обработчике массива через `.reduce`) часто ведёт себя неинтуитивно – проще писать обычный `for-of`.

05 Если в цикле нужны и параллельность, и сохранение порядка результата – стартуем все промисы синхронно, потом `await`-им в порядке индекса. Это другой паттерн, чем `Promise.all`.

код

// Последовательно – медленно

```
for (const id of ids) {  
  await fetchOne(id); // следующий ждёт  
}  
// 10 запросов по 200мс = ~2 секунды
```

// Параллельно – быстро

```
await Promise.all(ids.map(fetchOne));  
// 10 запросов одновременно = ~200мс
```

// `forEach + async` – типичная ошибка

```
ids.forEach(async (id) => {  
  await fetchOne(id);  
  // ошибки тут – unhandled rejection  
});  
// функция вернёт управление до завершения запросов
```

// Когда последовательно ОБЯЗАТЕЛЬНО

```
// пагинация по cursor – следующий зависит от текущего  
let cursor = null;  
do {  
  const page = await fetchPage(cursor);  
  cursor = page.nextCursor;  
  process(page.items);  
} while (cursor);
```

// Параллельно с сохранением порядка результатов

```
const tasks = ids.map(id => fetchOne(id)); // все стартуют  
for (const task of tasks) {  
  const data = await task; // в порядке  
  process(data);  
}
```

МОГУТ ДОСПРОСИТЬ

01 Когда нужен последовательный `await` – приведи реальный сценарий.

02 Что выведет `for-await-of` по массиву из 3 промисов?

03 Чем отличается `ids.map(asyncFn)` от `for-of-await` по поведению?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/async-await>

Когда лучше использовать `parallel execution`, а когда `sequential`?

ОПИСАНИЕ

Выбор между параллельным и последовательным выполнением — это баланс между скоростью, зависимостями данных, нагрузкой на ресурсы и риском гонок. Параллельно — быстрее, но требует, чтобы задачи были независимы и сервер выдержал такую нагрузку.

Последовательно — медленнее, зато проще: понятный порядок, отсутствие гонок, можно использовать результат предыдущего шага в следующем. Третий вариант — параллельно, но с лимитом одновременных задач — закрывает большую часть реальных кейсов с пачками работы.

КАК РАБОТАЕТ

01 Параллельно: задачи запускаются ВСЕ сразу, общий промис ждёт самой медленной. Реализация — `Promise.all` (все или ничего), `Promise.allSettled` (все, без падения), `Promise.race` (первый).

02 Последовательно: задача N стартует только после N-1. Реализация — `for + await`, `.then`-цепочка, `async generator`.

03 Лимитированная параллель: одновременно работает N задач, новые берутся из очереди по освобождению слота. Реализация — пул, `p-limit`, библиотечный `Bottleneck`.

04 Параллель экономит время: общая длительность $\approx$ длительность самой медленной задачи. Последовательно: сумма длительностей.

05 В Node параллельность ограничена пулом `libuv` для I/O (по умолчанию 4 потока на файловые/DNS-операции). HTTP-запросы идут через `event loop`, без блокировки.

ГДЕ ВСТРЕЧАЕТСЯ

01 Параллель — независимые задачи, общий результат собирается по приходе всех (загрузка юзера и постов одновременно, `batch`-запросы к разным API).

02 Последовательно — есть зависимость по данным (В нужен результат А), важен порядок изменений (последовательные записи в одну сущность), либо боимся `race condition`.

03 Лимит — большая пачка задач, которая может перегрузить сервер или сеть, если запустить всё разом. Скачивание/обработка файлов, `batch`-операции с API.

04 Если задача атомарна и быстрая — параллель. Если каждая может быть тяжёлой и долгой — лимит, чтобы не задушить `event loop`.

ПОДВОДНЫЕ КАМНИ

01 1000 параллельных `fetch` — гарантированный 429 или DDoS своего же бэкенда. На клиенте — забитая `connection pool`, висящие запросы.

02 Параллель повышает шанс `race condition`, если задачи касаются общего состояния — глобального кэша, файла на диске, `shared`-сервиса.

03 Последовательный `await` там, где задачи независимы — потерянное время и дорогая ошибка по перфомансу.

04 Lazy evaluation: `arr.map(asyncFn)` – это не вызов, это создание массива промисов; все они стартуют сразу. Если не понимать этого, легко удивиться скрытой параллельности.

05 В тестах: параллельные операции делают тесты flaky, потому что порядок завершения непредсказуем. Часто проще делать последовательно или мокать таймеры.

06 В UI параллельность через `Promise.all` без отмены – источник stale response (см. Q10).

код

// Параллель – независимые

```
const [user, posts] = await Promise.all([
  fetchUser(id),
  fetchPosts(id),
]);
```

// Последовательно – есть зависимость

```
const user = await fetchUser(id);
const posts = await fetchPostsBy(user.id);
// posts невозможно загрузить, не зная user.id
```

// Параллельно с лимитом – pool

```
// см. Q13 про реализацию пула
const results = await pool(urls, 5, fetchOne);
```

// Скрытая параллельность через `.map`

```
const tasks = ids.map(id => fetchOne(id));
// все промисы УЖЕ запустились, ждать их можно потом
const all = await Promise.all(tasks);
```

МОГУТ ДОСПРОСИТЬ

01 Сценарий: 50 файлов в S3, каждый ~10 МБ. Параллель, последовательно или с лимитом? Почему?

02 Что произойдёт, если делать `Promise.all` из 1000 fetch-ов?

03 Какой паттерн использовать, если порядок результатов важен, а скорость – тоже?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/async-await>

RACE CONDITION**STALE RESPONSE В REACT****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Связка из двух — теория и практика на одну боль. На неё спрашивают почти всегда, понимание должно быть железное.

ВОПРОСЫ ДНЯ

Q09 Что такое race condition в async-коде?

Q10 Как защититься от stale response в React-компоненте?

ВОПРОС Q09

Что такое race condition в async-коде?

ОПИСАНИЕ

Race condition – это когда исход операции зависит от того, какой из параллельных асинхронных запросов завершился первым, а гарантировать порядок мы не можем. В UI это часто проявляется как «прыгающие» данные: устаревший ответ, вернувшийся позже актуального, перезапишет свежие результаты. В бэкенд-коде это могут быть две одновременные записи в одну сущность, между которыми есть промежуток чтения, и обе пишут разный результат. Решения – отмена устаревших запросов (AbortController), счётчик версий с проверкой при ответе, или маркер последнего ввода для last-write-wins.

КАК РАБОТАЕТ

- 01 Параллельные запросы стартуют почти одновременно, но завершаются в случайном порядке: время ответа зависит от сети, нагрузки на сервер, перезапусков подов и пр.
- 02 Если их обработчики пишут в общее состояние (state, localStorage, DOM) – последний сработавший выигрывает. Это «last write wins».
- 03 Если последний сработавший – устаревший запрос, актуальные данные затираются, и пользователь видит неправильный результат. Самый частый пример – поиск-автоподсказки.
- 04 Стратегии защиты: 1) AbortController – отменить старые запросы при появлении нового; 2) счётчик версий – каждому запросу присвоить id, при ответе сравнить с актуальным id; 3) маркер последнего ввода (last-input-key) – обновлять только если ключ совпадает с текущим состоянием.
- 05 В React-эффектах это превращается в stale response – отдельный вопрос (Q10) про конкретную защиту в useEffect.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Поиск-автоподсказки: пользователь печатает «ab», потом «abc» – ответ на «ab» не должен затирает «abc». Один из самых частых примеров.
- 02 Переключение вкладок/фильтров – старая вкладка дописала свои данные после того, как уже открыта новая.
- 03 Серия записей одной сущности – порядок применения меняет результат. Часто видно в формах с автосохранением.
- 04 Кэш-обновления при множественных параллельных запросах за одним и тем же ресурсом.
- 05 Multi-tab: две вкладки одного приложения параллельно меняют сервер-состояние и пишут в localStorage.

ПОДВОДНЫЕ КАМНИ

- 01 Без явной защиты race condition воспроизводится только под нагрузкой и медленной сетью – в локальной разработке его легко не увидеть. Полезно эмулировать throttling в DevTools.

- 02 `AbortController` отменяет `fetch` на стороне клиента, но запрос на сервере уже мог начать работу. Если запрос делает побочные эффекты — отмена клиента не отменит их на сервере.
- 03 Простой `debounce` уменьшает частоту запросов, но не убирает гонку, если запрос таки запустился. Нужно сочетать `debounce` + один из методов защиты.
- 04 `Race condition` в `state-management` часто маскируется: `Redux/Zustand` не падают, просто пишут «не то». Проявляется как баг отображения, виноват `asunc`, обнаруживают его поздно.
- 05 В тестах `race condition` часто `flaky`: тест зелёный 9 из 10 запусков. Полезно специально моделировать порядок ответов через моки таймеров.

КОД

// Защита через `id`

```
let lastId = 0;
async function search(q) {
  const myId = ++lastId;
  const data = await api.search(q);
  if (myId !== lastId) return; // устарело
  setResults(data);
}
```

// Через `AbortController`

```
let ctrl;
async function search(q) {
  ctrl?.abort();
  ctrl = new AbortController();
  try {
    const data = await api.search(q, { signal: ctrl.signal });
    setResults(data);
  } catch (e) {
    if (e.name !== 'AbortError') throw e;
  }
}
```

// Маркер последнего ввода

```
let currentKey = '';
async function search(q) {
  currentKey = q;
  const data = await api.search(q);
  if (currentKey !== q) return;
  setResults(data);
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Как абортить `fetch` с таймаутом?
- 02 Когда `race condition` НЕ опасен — приведи пример.
- 03 Чем счётчик версий отличается от `AbortController` на практике?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/fetch-abort>

ВОПРОС Q10

Как защититься от stale response в React-компоненте?

ОПИСАНИЕ

Stale response – это когда асинхронный запрос внутри `useEffect` вернулся уже после того, как `props` или `state` поменялись, и его `setState` пишет устаревшие данные в актуальный компонент. Это конкретный React-кейс `race condition` (Q9). Самые распространённые решения – флаг `cancelled` в замыкании эффекта и `AbortController`. Оба вешаются на `cleanup`-функцию: при перезапуске эффекта (смена `id`, размонтирование) либо помечаем запрос как ненужный, либо отменяем его. Альтернатива – взять библиотеку уровня `react-query` или `SWR`, где это реализовано под капотом.

КАК РАБОТАЕТ

- 01 Эффект перезапускается при смене зависимости (например, `id`). React сначала вызывает `cleanup` предыдущего эффекта, потом запускает новый.
- 02 Старый асинхронный запрос (созданный в предыдущем эффекте) продолжает работать в фоне – отмены по умолчанию нет.
- 03 Если он вернётся ПОСЛЕ перезапуска эффекта, его `setState` запишет устаревшие данные в уже актуализированный компонент.
- 04 Решение 1 – флаг в замыкании эффекта: при `cleanup` ставим `cancelled = true`, перед `setState` проверяем. Самый простой и портативный способ.
- 05 Решение 2 – `AbortController`: пробрасываем `signal` в `fetch`, в `cleanup` вызываем `abort()`. Запрос отменяется на сетевом уровне, `.catch` ловит `AbortError`.
- 06 В React 18 со `Strict Mode` эффекты в `dev` запускаются дважды подряд – это намеренно ловит баги без `cleanup`. Если защититесь правильно, ничего не сломается.
- 07 `useEffect` не должен возвращать промис – его возвращаемое значение интерпретируется как `cleanup`-функция. Для `async` обычно объявляют локальную `async`-функцию и вызывают её внутри.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая загрузка данных по `props` (`id`, фильтр) внутри `useEffect` обязана защититься от stale.
- 02 Поиск с автоподсказками внутри React-компонента – канонический пример.
- 03 Переключение вкладок/маршрутов с фоновой загрузкой – другой частый кейс.
- 04 Эффекты, которые запускают `setInterval/WebSocket/observer` – тоже требуют `cleanup`, но проблема немного другая (утечка ресурса).

ПОДВОДНЫЕ КАМНИ

- 01 Просто `async useEffect` нельзя – `useEffect` не должен возвращать промис. Нужно объявлять локальную `async`-функцию внутри.
- 02 Без `cleanup` React всё равно вызовет `setState` на размонтированном компоненте – в новых версиях React это уже не `warning`, но логика может сломаться, а `memory leak` –

остаться.

03 `AbortController` отменяет только `fetch` (и любой совместимый с `AbortSignal` API). Если внутри ещё какие-то побочные действия (`WebSocket`, таймеры) — их надо отменять отдельно.

04 Библиотеки уровня `react-query` / `SWR` закрывают это под капотом — `stale`-маркеры, ключи-кэши, дедуп одновременных запросов. Если в проекте что-то такое уже есть — обычно проще пользоваться им, чем писать руками.

05 В Strict Mode React 18 эффект запускается дважды — поэтому если `cleanup` не защищает корректно, баг будет виден в `dev` и спрячется в `prod` (где `double-effect` не запускается).

код

// Cleanup-флаг (самое простое и портатбельное)

```
useEffect(() => {
  let cancelled = false;
  fetchUser(id).then(u => {
    if (!cancelled) setUser(u);
  });
  return () => { cancelled = true; };
}, [id]);
```

// `AbortController` (отменяем по сети)

```
useEffect(() => {
  const ctrl = new AbortController();
  fetch(`/api/user/${id}`, { signal: ctrl.signal })
    .then(r => r.json())
    .then(setUser)
    .catch(err => {
      if (err.name !== 'AbortError') throw err;
    });
  return () => ctrl.abort();
}, [id]);
```

// Асинх-функция внутри эффекта

```
useEffect(() => {
  let cancelled = false;
  async function load() {
    try {
      const u = await fetchUser(id);
      if (!cancelled) setUser(u);
    } catch (e) {
      if (!cancelled) setError(e);
    }
  }
  load();
  return () => { cancelled = true; };
}, [id]);
```

МОГУТ ДОСПРОСИТЬ

01 Почему `useEffect` не должен возвращать промис?

02 Как `react-query` решает эту проблему под капотом?

- 03 Что покажет Strict Mode в React 18, если забыть cleanup?
- 04 Когда выбрать AbortController, а когда – флаг cancelled?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/fetch-abort>

ПРАКТИКА**RETRY + BACKOFF****ABORT FETCH****CONCURRENCY****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Кодинг-вопросы. Желательно руками прописать `retry` и `pool` с лимитом параллелизма.

ВОПРОСЫ ДНЯ

Q07 Как реализовать `retry` с `backoff`?

Q08 Как отменить `fetch`-запрос?

Q13 Как ограничить `concurrency` для пачки `async`-задач?

ВОПРОС Q07

Как реализовать retry с backoff?

ОПИСАНИЕ

Retry – автоматический повтор неудачной операции. Backoff – нарастающая задержка между попытками. Самая частая комбинация – exponential backoff с jitter: ждать 200, 400, 800 миллисекунд и так далее, плюс случайное отклонение, чтобы клиенты не штормили сервер синхронно. Минимум – лимит попыток и таймаут на каждую попытку. В реальной системе ещё проверять класс ошибки: 5xx и сетевые – повторять; 4xx (кроме 429) – обычно нет; 429 – уважать Retry-After заголовок. Без backoff простой retry превращается в DDoS своего сервера в момент, когда он только начинает восстанавливаться.

КАК РАБОТАЕТ

- 01 Параметры: maxAttempts (сколько попыток всего), baseDelay (стартовая задержка), factor (множитель, обычно 2), jitter (случайные 0-30% от задержки), maxDelay (потолок).
- 02 Основной цикл: `try { await fn() } catch { ждать → следующая попытка }`. На последней попытке – выкинуть ошибку наружу.
- 03 Задержка считается: `min(baseDelay × factor^(attempt-1), maxDelay) + jitter`. Jitter – это `math.random() × delay × jitterFactor`.
- 04 Backoff бывает разный: linear (200, 400, 600), exponential (200, 400, 800, 1600), polynomial. Самый частый и стабильный – exponential с jitter.
- 05 Опционально – оборачивать каждую попытку в таймаут, чтобы не зависнуть на одной. Через `AbortSignal.timeout(ms)` или race с таймаут-промисом.
- 06 Лучше передавать в retry функцию, а не уже-запущенный промис: промис нельзя «перезапустить». Поэтому сигнатура – `withRetry(() => fetch(url), opts)`.
- 07 Уважение Retry-After: если сервер вернул 429 или 503 с этим заголовком, использовать его значение вместо собственного backoff.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые сетевые запросы, где сервер может временно лежать или ответить 5xx. По умолчанию retry имеет смысл для GET (идемпотентных), для POST – осторожно.
- 02 Очереди задач, фоновые отправки (аналитика, отложенные действия) – retry с backoff почти всегда обязателен.
- 03 Подписки на realtime-каналы – реконнект с backoff после обрыва. Здесь exponential backoff критичен, иначе после массового разрыва все клиенты ломанутся обратно одновременно.
- 04 В библиотеках – react-query (через retry config), axios-retry, p-retry. Часто проще взять готовое.

ПОДВОДНЫЕ КАМНИ

- 01 Не все ошибки одинаково повторяемы. 5xx и сетевые – повторять. 4xx (кроме 429) – обычно нет, иначе долбим бесполезно. POST с побочными эффектами – обычно НЕЛЬЗЯ повторять, если нет идемпотентности.
- 02 Без jitter синхронный «retry storm» от множества клиентов после общего сбоя усугубляет ситуацию. Это известный паттерн распределённых систем – thundering herd.
- 03 Без таймаута на попытку retry зависнет на медленной сети.
- 04 Если сервер вернул Retry-After – лучше уважать его, а не считать backoff заново. Иначе попадаем в новый штрафной круг.
- 05 MaxAttempts слишком большой – пользователь ждёт минутами; слишком маленький – пропускаем кратковременные сбои. Эмпирика: 3-5 попыток с базой 200мс хватает для большинства сценариев.
- 06 Retry внутри пула с лимитом параллелизма (см. Q13) может удвоить эффективную параллельность – это надо учитывать.
-

код

// Базовый retry с exponential backoff и jitter

```
async function withRetry(fn, {
  maxAttempts = 5, base = 200, factor = 2,
  max = 5000, jitter = 0.3,
} = {}) {
  let attempt = 0;
  while (true) {
    try {
      return await fn();
    } catch (e) {
      attempt++;
      if (attempt >= maxAttempts) throw e;
      const delay = Math.min(
        base * factor ** (attempt - 1), max
      );
      const jit = delay * Math.random() * jitter;
      await new Promise(r => setTimeout(r, delay + jit));
    }
  }
}
```

// withRetry(() => fetch('/api'))

// С учётом класса ошибки

```
function shouldRetry(err) {
  if (err.name === 'AbortError') return false;
  const status = err.status ?? 0;
  if (status === 429) return true;
  if (status >= 500) return true;
  if (status >= 400) return false;
  return true; // network errors
}

// в catch перед перезапуском:
// if (!shouldRetry(e)) throw e;
```

// Уважаем Retry-After

```
function retryAfter(res) {
```



```
const h = res.headers.get('Retry-After');
if (!h) return null;
// секунды или HTTP-date
const seconds = Number(h);
if (!Number.isNaN(seconds)) return seconds * 1000;
const ts = Date.parse(h);
return Number.isNaN(ts) ? null : ts - Date.now();
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Как комбинировать retry с AbortController, чтобы пользователь мог отменить?
- 02 Что делать, если сервер вернул Retry-After?
- 03 Какие методы HTTP безопасно повторять, а какие нет?
- 04 В чём идея thundering herd и как jitter её снижает?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/fetch-abort>

Как отменить fetch-запрос?

ОПИСАНИЕ

Через `AbortController` – стандартный API. Создаём контроллер, передаём его `signal` в `fetch`, когда нужно отменить – `controller.abort()`. `fetch` упадёт с ошибкой типа `AbortError`, её обрабатывают отдельно от обычных сетевых ошибок. Один контроллер можно использовать для нескольких запросов и отменить их все одним вызовом – это удобно при `cleanup` целого экрана. Современный браузерный API также даёт `AbortSignal.timeout(ms)` – встроенный способ сделать запрос с таймаутом без ручной возни с `setTimeout`.

КАК РАБОТАЕТ

- 01 `AbortController` создаётся одним вызовом: `const ctrl = new AbortController()`. У него есть свойство `signal` – это `AbortSignal`.
- 02 `AbortSignal` – `read-only` объект, его передают в `fetch` как опцию: `fetch(url, { signal: ctrl.signal })`.
- 03 `controller.abort()` переводит `signal.aborted` в `true`, эмитит событие `abort` и заставляет все наблюдатели (`fetch` и др.) среагировать.
- 04 `fetch` на отменённом сигнале мгновенно реджектится с ошибкой, у которой `name === 'AbortError'`. Это НЕ ошибка сети, и обычно её не нужно показывать пользователю.
- 05 Один сигнал можно передать в несколько `fetch` – отменятся все одним вызовом `abort()`. Это удобный способ сделать «отмена целого экрана».
- 06 Современный API: `AbortSignal.timeout(ms)` – возвращает `signal`, который автоматически `abort`-ится через `ms` миллисекунд. Доступен в современных браузерах и Node 17+.
- 07 `AbortSignal.any([s1, s2])` – комбинирует несколько сигналов в один: общий `abort` случается при `abort` любого из исходных.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Поиск с автоподсказками – отменять прошлый запрос при новом вводе, чтобы не получить `stale response` (Q9, Q10).
- 02 `Cleanup` в `useEffect` – отменять при размонтировании или смене `props`. Эффект, не отменивший запрос, потенциально пишет в неактуальный компонент.
- 03 Ручная кнопка «Отменить» в UI на длинных операциях – загрузка больших файлов, экспорт.
- 04 Таймаут на запрос: `AbortSignal.timeout(5000)`. Сервер может висеть бесконечно, клиент не должен.
- 05 Маршрутизация: при переходе на другой экран отменить все запросы старого. Часто организуют через один `AbortController` на маршрут.

ПОДВОДНЫЕ КАМНИ

01 `AbortError` – НЕ ошибка сети, её надо отдельно фильтровать в `catch`, иначе она попадёт в общий `error`-обработчик и может, например, показать `toast` «ошибка сети» на безобидную отмену.

02 Отмена работает только на уровне клиента – сервер уже мог получить запрос и обработать его. Если запрос делает побочные эффекты, `abort` их не отменит.

03 `AbortController` одноразовый: после `abort()` он навсегда `aborted`, для следующего запроса нужен новый. Поэтому в эффекте контроллер создают на каждом запуске.

04 `AbortSignal.timeout` появился относительно недавно – в старых средах нужно делать вручную через `setTimeout` + `abort`. Это известный паттерн и его легко написать самому.

05 Если повесить `fetch` на `signal`, который УЖЕ `aborted` – `fetch` сразу отреджектится. Это нормальное поведение, но иногда удивляет.

06 Не все API-обёртки прокидывают `signal`: некоторые библиотеки игнорируют его, и отмена не работает. При выборе HTTP-клиента стоит проверять.

КОД

// Базовая отмена

```
const ctrl = new AbortController();
fetch('/api', { signal: ctrl.signal })
  .then(r => r.json())
  .catch(err => {
    if (err.name === 'AbortError') return;
    throw err;
  });
// в любой момент:
ctrl.abort();
```

// Таймаут через `signal`

```
const signal = AbortSignal.timeout(5000);
try {
  const res = await fetch('/api', { signal });
} catch (e) {
  if (e.name === 'TimeoutError' || e.name === 'AbortError') {
    // прошёл таймаут
  }
}
```

// Один сигнал – несколько запросов

```
const ctrl = new AbortController();
const all = await Promise.all([
  fetch('/a', { signal: ctrl.signal }),
  fetch('/b', { signal: ctrl.signal }),
  fetch('/c', { signal: ctrl.signal }),
]);
// ctrl.abort() – отменит все три
```

// Свой `timeout-helper` для старых сред

```
function withTimeout(promise, ms) {
  const ctrl = new AbortController();
  const t = setTimeout(() => ctrl.abort(), ms);
  return Promise.resolve(promise(ctrl.signal))
    .finally(() => clearTimeout(t));
}
```

```
// withTimeout(signal => fetch('/api', { signal }), 5000)
```

МОГУТ ДОСПРОСИТЬ

- 01 Как отменить группу запросов одним сигналом?
 - 02 Можно ли передать `signal` не только в `fetch`?
 - 03 Что произойдёт, если повесить `fetch` на уже `aborted-signal`?
 - 04 Как сделать таймаут без `AbortSignal.timeout`?
-

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/fetch-abort>

ВОПРОС Q13

Как ограничить concurrency для пачки async-задач?

ОПИСАНИЕ

Concurrency limit – это компромисс между Promise.all (всё сразу) и последовательным выполнением. Запускается одновременно не больше N задач; новые берутся по освобождению слота. Защищает бэкэнд от перегрузки и снижает риск 429 / забитого connection pool. Простейший паттерн – пул из N воркеров: каждый воркер в цикле берёт следующую задачу из общего курсора и await-ит её. Альтернатива – chunk-разбиение, но оно медленнее, потому что один тормозящий task в чанке держит остальные слоты простаивающими.

КАК РАБОТАЕТ

- 01 Простой пул: создаём N воркеров (async-функций), каждый в цикле берёт следующую задачу из общего курсора и await-ит её.
- 02 Курсор – переменная-индекс. Каждый воркер увеличивает её и берёт элемент массива по этому индексу. Когда курсор догнал длину массива – воркер выходит из цикла.
- 03 Promise.all над N воркерами ждёт, пока все они опустошат очередь.
- 04 Альтернативный паттерн – chunk: разбить массив на куски по N, обрабатывать Promise.all чанк за чанком. Минус – медленные таски тормозят весь чанк, в это время остальные слоты простаивают.
- 05 Если результаты нужно сохранить в исходном порядке – записывать их в массив по индексу задачи, а не push-ить в общий результат.
- 06 Готовые либы: p-limit, p-мар, async-pool, Bottleneck. В react-query сетевая конкуренция настраивается опцией.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Скачивание/обработка большой пачки файлов или URL.
- 02 Массовые операции с API, у которого есть rate limit.
- 03 Параллельный resize/обработка картинок на клиенте – иначе браузер залипнет.
- 04 В SSR / static generation – параллельная сборка страниц с лимитом, чтобы не задушить процесс.
- 05 Краулеры и парсеры – почти всегда с лимитом, чтобы не получить бан и не нагрузить целевой сервер.

ПОДВОДНЫЕ КАМНИ

- 01 Promise.all без лимита запускает ВСЁ сразу – частая причина 429 и утечек памяти на больших пачках. На 10000 запросах это становится явной проблемой.
- 02 Чанк-разбиение проще, но N-1 медленных тасков на чанк держат остальные слоты простаивающими – пул эффективнее.
- 03 Лимит выбирается под конкретный бэкэнд: 5-10 для browser-fetch обычно ок, для тяжёлых операций может быть и 1-2.

04 Если внутри пула ещё и `retry` – можно раздуть фактическую параллельность; нужно учитывать ($\text{effective concurrency} = \text{limit} \times \text{среднее число retry}$).

05 Если задача может зависнуть – пул зависнет тоже. Нужен таймаут на каждый `task`.

06 При ошибке в `task` пул должен не падать целиком, а пропускать и идти дальше (или возвращать как `rejected` – зависит от семантики).

код

// Простейший pool с сохранением порядка

```
async function pool(items, n, worker) {
  const results = new Array(items.length);
  let i = 0;
  const run = async () => {
    while (i < items.length) {
      const idx = i++;
      results[idx] = await worker(items[idx], idx);
    }
  };
  await Promise.all(
    Array.from({ length: Math.min(n, items.length) }, run)
  );
  return results;
}
// const data = await pool(urls, 5, fetchOne);
```

// Pool с `allSettled`-семантикой

```
async function poolSafe(items, n, worker) {
  const results = new Array(items.length);
  let i = 0;
  const run = async () => {
    while (i < items.length) {
      const idx = i++;
      try {
        results[idx] = { status: 'fulfilled',
          value: await worker(items[idx]) };
      } catch (e) {
        results[idx] = { status: 'rejected', reason: e };
      }
    }
  };
  await Promise.all(
    Array.from({ length: Math.min(n, items.length) }, run)
  );
  return results;
}
```

// Chunk-вариант (для сравнения)

```
async function chunkRun(items, n, worker) {
  const results = [];
  for (let i = 0; i < items.length; i += n) {
    const chunk = items.slice(i, i + n);
    const part = await Promise.all(chunk.map(worker));
    results.push(...part);
  }
}
```

```
    return results;
  }
  // проще, но медленнее на разнородных задачах
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем pool лучше chunk-разбиения по N?
- 02 Как добавить retry внутри pool, не теряя лимита?
- 03 Как ограничить общую длительность работы пула таймаутом?

ПОЛНАЯ СТАТЬЯ

<https://learn.javascript.ru/promise-api>

DEBOUNCE / THROTTLE**rAF / rIC****STARVATION****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Закрепление и тайминговые тонкости. На этот день не оставлять новую теорию — повторение и пазлы.

ВОПРОСЫ ДНЯ

- Q14 Что такое debounce и throttle?
- Q15 Чем debounce отличается от throttle?
- Q16 Где лучше использовать requestAnimationFrame?
- Q17 Где лучше использовать requestIdleCallback?
- Q18 Что такое starvation в контексте event loop?

ВОПРОС Q14

Что такое debounce и throttle?

ОПИСАНИЕ

Это две разные стратегии «реже вызывать функцию» при потоке событий — типа набора в инпуте, скролла, перетаскивания, ресайза окна. Debounce ждёт паузы и выполняет один раз после неё: «тихо N мс — вызвать». Throttle разрешает вызовы не чаще, чем раз в N мс: «прошло N — можно опять». Внешне они похожи (оба «реже вызывать»), но реакция на поток событий противоположная: debounce схлопывает поток в одну точку в конце; throttle выдаёт регулярные вспышки в течение потока. Реализация обоих — 5-10 строк кода, но в библиотеках типа lodash они с поддержкой leading/trailing-флагов и cancel-метода.

КАК РАБОТАЕТ

- 01 Debounce: на каждое событие сбрасываем таймер и ставим новый. Если событий нет N мс — таймер срабатывает, функция выполняется. Реализация — один `setTimeout` + `clearTimeout`.
- 02 Throttle: запоминаем время последнего вызова. Новые события в окне `[last; last+N)` игнорируются. Реализация — флаг или timestamp последней инвокации.
- 03 У обоих есть варианты `leading` / `trailing` — вызывать в начале окна, в конце или в обоих случаях. Lodash называет их `{leading: true, trailing: true}`.
- 04 Cancel: отменить отложенный вызов. Полезно при `cleanup` в React-эффектах.
- 05 Flush: принудительно выполнить отложенный вызов сейчас, не дожидаясь таймера. Например, при `submit` формы.
- 06 В отличие от throttle с фиксированным окном, есть `rolling throttle` (`token bucket`), но он сложнее и обычно избыточен.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Debounce — поиск по вводу (искать после паузы в наборе), валидация формы (не валидировать на каждый символ), ресайз окна с дорогим пересчётом, автосейв черновика.
- 02 Throttle — обработка скролла (обновлять позицию хедера), перетаскивание мыши (двигать элемент), аналитика прогресса видео, обновление карты при панорамировании.
- 03 Часто берут готовые из lodash, но руками — 5-10 строк. В React — полезно завернуть в `useMemo`, иначе пересоздаётся на каждом рендере.
- 04 На сервере debounce-style паттерны видны при `agregation` событий (накапливать N мс, потом отправить пачкой).

ПОДВОДНЫЕ КАМНИ

- 01 Debounce может «никогда не выполниться», если события идут непрерывно с интервалом меньше N мс. Это *by design*, но иногда удивительно.
- 02 Throttle «теряет» события внутри окна — это *by design*, но `trailing`-вызов часто забывают, и последнее событие после серии может пропасть.

- 03 В React: создавать debounce/throttle ВНУТРИ компонента – пересоздаётся на каждый рендер, теряет внутреннее состояние (таймер). Заворачивать в useMemo / useCallback / useDebounceCallback.
- 04 Если в throttle колбэк сам что-то setState-ит и вызывает новый рендер – есть риск заикливания.
- 05 В debounce закрывание (closure) над свежими данными – нюанс: внутри замыкания «свежие» переменные при каждом вызове возвращаемой функции. Это автоматически работает, но при использовании лямбды через bind/this можно потерять.
- 06 На событиях с passive listeners (scroll/touch) лучше throttle, чтобы не мешать прокрутке.

код

// Debounce – простая версия

```
function debounce(fn, ms) {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), ms);
  };
}
```

// Throttle – простая версия

```
function throttle(fn, ms) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) {
      last = now;
      fn(...args);
    }
  };
}
```

// Debounce с cancel и flush

```
function debounce(fn, ms) {
  let t, lastArgs;
  function debounced(...args) {
    lastArgs = args;
    clearTimeout(t);
    t = setTimeout(() => fn(...lastArgs), ms);
  }
  debounced.cancel = () => clearTimeout(t);
  debounced.flush = () => { clearTimeout(t); fn(...lastArgs); };
  return debounced;
}
```

// Применение в React

```
const debouncedSearch = useMemo(
  () => debounce((q) => api.search(q).then(setItems), 300),
  [setItems]
);
useEffect(() => () => debouncedSearch.cancel?(), [debouncedSearch]);
```

МОГУТ ДОСПРОСИТЬ

- 01 Куда деть последний вызов после потока событий в throttle?
- 02 Как собрать leading/trailing варианты?
- 03 Когда лучше throttle с trailing, чем debounce?

Чем debounce отличается от throttle?

ОПИСАНИЕ

Оба ограничивают частоту вызовов, но реагируют на поток по-разному. Debounce – «ждём пауз»: пока поток идёт, ничего не вызывается; срабатывает один раз после N мс тишины. Итог – один итоговый вызов на всю серию событий. Throttle – «режем поток на интервалы»: вызывает не чаще, чем раз в N мс, независимо от того, идут события или нет; срабатывает в течение потока. Итог – несколько вызовов на серию, с шагом N. Эта разница меняет всё: для коротко-вспышечного потока (ввод, ресайз) нужен debounce; для непрерывного (скролл, drag) – throttle, иначе debounce не работает вообще, пока пользователь катает.

КАК РАБОТАЕТ

- 01 Debounce: пока поток идёт – функция НЕ вызывается. Срабатывает ровно один раз после N мс тишины. Итог – один итоговый вызов на серию событий любой длины.
- 02 Throttle: вызывает не чаще, чем раз в N мс. Срабатывает в течение потока, не ждёт окончания. Итог – несколько вызовов на серию, с шагом N.
- 03 В debounce таймер постоянно сбрасывается; в throttle – не сбрасывается, проверяется условие «прошло ли N с прошлого вызова».
- 04 Если события идут с интервалом $\geq N$ мс, debounce и throttle ведут себя одинаково – каждый раз вызывают функцию.
- 05 При комбинировании leading + trailing throttle становится ближе к debounce – есть и реакция в начале, и финальный вызов после серии.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Поток коротко-вспышечный (ввод, ресайз) → debounce: результат нужен один и только после паузы.
- 02 Поток непрерывный (скролл, drag) → throttle: иначе debounce не работает вообще, пока пользователь катает.
- 03 На onResize окна – обычно debounce, чтобы не пересчитывать лэйаут на каждый пиксель в процессе ресайза.
- 04 На onScroll – throttle, чтобы плавно обновлять навбар по ходу прокрутки.
- 05 Иногда нужно одновременно: throttle с trailing-вызовом ≈ компромисс между «реагировать в потоке» и «дать финальный ответ».

ПОДВОДНЫЕ КАМНИ

- 01 Если выбрать debounce для скролла – функция не вызовется, пока пользователь не остановится. Эффект «UI не реагирует».
- 02 Если выбрать throttle для поиска по вводу – будет несколько лишних запросов вместо одного. Дороже по сети.

03 На onResize окна – обычно debounce, но если хочется плавности (например, отзывчивая верстка) – throttle.

04 Throttle с trailing включён только при flag – иначе последний вызов может потеряться. Это известная тонкость lodash.

05 Для двойного клика, taps и подобного – НИ ТО, НИ ДРУГОЕ, там нужен другой паттерн (детектор события).

КОД

// Когда что выбрать

```
// debounce – поиск
input.oninput = debounce(
  e => api.search(e.target.value), 300
);

// throttle – скролл
window.onscroll = throttle(
  () => updateNavbar(), 100
);
```

// Что увидит пользователь

```
// поток: 'a','b','c','d', пауза 350мс, 'e'
// debounce(300): один вызов с 'd', потом один с 'e'
// throttle(300): вызовы с 'a' (или какой первый),
//                потом каждые ~300мс, потом с 'e' если
//                trailing включён
```

МОГУТ ДОСПРОСИТЬ

01 Что выбрать для onResize окна – debounce или throttle?

02 Что произойдёт в input-debounce, если ms = 0?

03 Можно ли реализовать throttle через debounce и наоборот?

Где лучше использовать requestAnimationFrame?

ОПИСАНИЕ

`requestAnimationFrame` синхронизирует выполнение кода с частотой рендера экрана. Колбэк вызывается перед следующим кадром (обычно ~60 раз в секунду, на 120-герцовых экранах — 120). `rAF` идеален для всего, что меняет визуал: анимации `position/transform/opacity`, перерисовка `canvas/WebGL`, плавный скролл, обновление DOM в ответ на тяжёлые события. `setTimeout` для анимаций даёт дёргание — он не выровнен по кадрам. Дополнительный плюс `rAF` — браузер сам останавливает его в скрытой вкладке, экономя CPU и батарею. В Node `rAF` нет (там нет рендера), но есть альтернативы вроде `requestAnimationFrame-полифиллов` и `scheduler-API`.

КАК РАБОТАЕТ

- 01 `rAF(fn)` — `fn` вызовется перед следующим `paint`. Без явной задержки в мс, привязка идёт к кадровой частоте.
- 02 Интервал ≈ 16.7 мс при 60 fps, на 120-герцовых экранах ≈ 8.3 мс. Реальные интервалы могут варьироваться: если кадр не успел отрисоваться, следующий `rAF` задержится.
- 03 Браузер пропускает `rAF` в скрытых вкладках — экономит CPU и батарею. После возвращения вкладки происходит один «догоняющий» `rAF`.
- 04 Колбэки `rAF` выполняются ДО `layout/paint` в кадре. Поэтому изменения, сделанные внутри `rAF`, попадают в этот же кадр.
- 05 `rAF` получает в аргумент `DOMHighResTimeStamp` — точное время начала кадра. Используется для считывания `delta`-времени в анимациях.
- 06 Чтобы анимация продолжалась — внутри `rAF` вызывают `rAF` снова. Чтобы остановить — просто не вызывают (или используют `cancelAnimationFrame` с сохранённым `id`).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Анимации `position/transform/opacity` на JS — апдейт визуала внутри `rAF`.
- 02 Перерисовка `canvas / WebGL` — игровые тики, графики, симуляции.
- 03 Скролл-обработчики — внутри `scroll handler` минимум, апдейт UI в `rAF` (через флаг `pending`).
- 04 Batch-обновления DOM, чтобы не делать `reflow` на каждый чих и попасть в один кадр.
- 05 Замер геометрии после изменения DOM — иногда дешевле подождать `rAF`, чтобы браузер успел сделать `layout` без принудительного `reflow`.

ПОДВОДНЫЕ КАМНИ

- 01 Если можно решить CSS-анимацией/`transition` — лучше так: это полностью на стороне композитора, без JS-потока.
- 02 `rAF` не идёт в скрытой вкладке — анимация замёрзнет до возврата. Если нужна работа в фоне — либо `setTimeout` (и принять ограничения), либо `Web Worker`.

- 03 rAF дёргает функцию максимум раз за кадр, но если внутри тяжёлая работа – кадр всё равно пропадёт.
- 04 Чтобы анимация не висела вечно – внутри rAF проверяем условие и не перезапускаем колбэк.
- 05 Не путать с requestIdleCallback (Q17): rIC – для невизуала, rAF – для визуала.
- 06 В тестах rAF часто мокают через jest.useFakeTimers – иначе тесты будут зависеть от реального времени и flaky.

КОД

// Тяжёлый scroll-handler

```
let pending = false;
window.addEventListener('scroll', () => {
  if (pending) return;
  pending = true;
  requestAnimationFrame(() => {
    updateUI();
    pending = false;
  });
});
```

// Простая анимация

```
let x = 0;
function tick() {
  el.style.transform = `translateX(${x++}px)`;
  if (x < 200) requestAnimationFrame(tick);
}
requestAnimationFrame(tick);
```

// Cancel при unmount (например, в useEffect)

```
useEffect(() => {
  let id;
  function tick(t) {
    update(t);
    id = requestAnimationFrame(tick);
  }
  id = requestAnimationFrame(tick);
  return () => cancelAnimationFrame(id);
}, []);
```

// Замер delta-времени

```
let last = performance.now();
function tick(t) {
  const dt = t - last;
  last = t;
  move(speed * dt / 1000);
  requestAnimationFrame(tick);
}
requestAnimationFrame(tick);
```

МОГУТ ДОСПРОСИТЬ

- 01 Что лучше для анимации – CSS transition или JS-rAF?
- 02 Что произойдёт с rAF в неактивной вкладке?

ВОПРОС Q17

Где лучше использовать `requestIdleCallback`?

ОПИСАНИЕ

`requestIdleCallback` вызывает колбэк, когда у браузера есть свободное время между кадрами. Ему передаётся объект `deadline` с методом `timeRemaining()` – сколько ещё миллисекунд можно поработать без того, чтобы пропустить следующий кадр. rIC подходит для всего, что НЕ срочно и НЕ визуально: аналитика, `prefetch` следующих экранов, фоновые расчёты, прогрев кэша, очистка устаревших данных. Для UI и анимаций он не подходит: rIC может задержаться на любое время или вообще не вызваться, если страница активно работает. Можно задать `timeout` – тогда браузер вызовет колбэк принудительно, но без гарантии `idle`-состояния.

КАК РАБОТАЕТ

- 01 `requestIdleCallback(cb, {timeout})` – `cb` вызовется в `idle`-период между кадрами. Возвращает `id`, который можно отменить через `cancelIdleCallback`.
- 02 `deadline.timeRemaining()` – сколько миллисекунд ещё можно работать до следующего кадра. Полезно делить большую работу на куски, проверяя бюджет.
- 03 `deadline.didTimeout` – `true`, если `cb` вызвался по таймауту, а не из-за реального `idle`. В этом случае `timeRemaining()` вернёт минимум, и стоит отложить тяжёлую работу.
- 04 Если за `timeout idle` не нашлось – `cb` вызовется принудительно с `didTimeout = true`. Иначе ждёт реальный `idle`.
- 05 В Safari rIC долго не было – сейчас есть, но не везде. Полифилл через `setTimeout(fn, 1)` даёт похожее поведение, но без гарантий бюджета.
- 06 Связка с rAF: запустить тяжёлое из rAF – заблокировать кадр; запустить из rIC – отложить до свободного слота. Иногда комбинируют: внутри rAF планируют rIC.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Аналитика, отправка телеметрии – не блокировать UI важным.
- 02 Pre-fetch / pre-render следующих экранов – пользователь вряд ли заметит, но переход станет мгновенным.
- 03 Фоновые вычисления, прогрев кэша, индексирование. Например, построение индекса поиска по локальным данным.
- 04 Очистка устаревших данных, low-priority фоновая работа: удаление старых записей из IndexedDB, ротация логов.
- 05 Аккуратная инициализация модулей, которые не нужны сразу.

ПОДВОДНЫЕ КАМНИ

- 01 Не подходит для UI: rIC может задержаться на любое время и вообще не вызваться, если страница занята. Анимации и интерактив – только rAF / rAFs.
- 02 Если внутри rIC запустить тяжёлый цикл без оглядки на `timeRemaining()` – заблокируется тот же UI, ради которого затевалось.

- 03 В Safari работает не везде – нужен полифилл через `setTimeout`.
- 04 Не путать с `rAF`: `rAF` – для визуала, `rIC` – для невизуала. Это разные инструменты.
- 05 `didTimeout = true` означает, что бюджета нет – нельзя жадно работать. Лучше переслать задачу обратно через `rIC`.
- 06 В Node нет `rIC` – там аналог `setImmediate` (макрозадача в фазе `check`) и `schedule.postTask` с приоритетами.

КОД

// Фоновое индексирование

```
function processChunks(chunks) {  
  function step(deadline) {  
    while (chunks.length && deadline.timeRemaining() > 5) {  
      processOne(chunks.pop());  
    }  
    if (chunks.length) requestIdleCallback(step);  
  }  
  requestIdleCallback(step);  
}
```

// Аналитика без блокировки

```
function sendAnalytics(payload) {  
  requestIdleCallback(  
    () => navigator.sendBeacon('/log', JSON.stringify(payload)),  
    { timeout: 2000 }  
  );  
}
```

// Polyfill для отсутствующего `rIC`

```
const idle = window.requestIdleCallback ||  
  function (cb, { timeout = 200 } = {}) {  
    return setTimeout(() => cb({  
      didTimeout: false,  
      timeRemaining: () => Math.max(0, 50 - 0),  
    }), 1);  
  };
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем `rIC` отличается от `setTimeout(0)` на практике?
- 02 Что произойдёт, если внутри `rIC` запустить тяжёлый цикл?
- 03 Как комбинировать `rIC` и `rAF`?

ВОПРОС Q18

Что такое starvation в контексте event loop?

ОПИСАНИЕ

Starvation (голодание) — это когда какие-то задачи в очереди не получают шанса выполниться, потому что более приоритетные задачи постоянно занимают цикл. В JS это обычно следствие того, что более «верхняя» очередь не пустеет: бесконечно пополняемые микрозадачи блокируют макрозадачи и рендер, тяжёлый синхронный `for` блокирует вообще всё. В Node ещё бывает starvation между фазами event loop из-за злоупотребления `process.nextTick`, который имеет приоритет выше всех микрозадач. Лечится разбивкой на чанки и переходом на менее приоритетный тип задачи (макро вместо микро, `setImmediate` вместо `nextTick`).

КАК РАБОТАЕТ

- 01 В браузере приоритет такой: синхронный код > микрозадачи > rAF > макрозадачи > рендер-фаза.
- 02 Если микрозадача добавляет ещё микрозадачу — макро и рендер встанут (см. Q5). Это и есть микро-старвейшен.
- 03 Если в одной макрозадаче тяжёлый синхронный `for` — не успевает ничего, включая клавиатуру и мышь. Это макро-старвейшен.
- 04 В Node.js `process.nextTick` идёт ПЕРЕД микрозадачами; рекурсивный `nextTick` душит всё остальное, включая `.then`-цепочки и I/O.
- 05 Лечение: делить работу на чанки через `setTimeout` / `setImmediate` / `scheduler.postTask` и давать event loop передышку между ними.
- 06 В современных браузерах есть `scheduler.postTask` с приоритетами `user-blocking` / `user-visible` / `background` — это явный API, чтобы делать «низкоприоритетную» работу без старвейшена.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Знание помогает дебажить «всё работает, а UI стоит» в браузере. Симптом — JS грузит CPU 100%, во вкладке Performance видны длинные блоки JS без зазоров под рендер.
- 02 В Node — следить, чтобы `process.nextTick` не превращался в петлю. Особенно при стрим-обработке: `nextTick` на каждый chunk без передышки = блок event loop.
- 03 При параллельной обработке большой пачки задач — ставить лимит и давать передышку, иначе можно задушить I/O.
- 04 В играх / симуляциях — балансировать тяжёлую работу между rAF (визуал) и rIC (невизуал), чтобы не голодал ни тот, ни другой.

ПОДВОДНЫЕ КАМНИ

- 01 Starvation не валит JS-код — он работает, и профайлер показывает CPU. Снаружи это выглядит как «зависший UI», а изнутри — как корректная работа без проблем.

02 Микро-старвейшен трудно поймать — это не очевидный `while(true)`. Полезно искать рекурсивные `.then` на одну и ту же функцию.

03 `scheduler.postTask` с приоритетом `'background'` помогает, но поддерживается ещё не везде — нужен фолбэк.

04 В Node старвейшен через `nextTick` может выглядеть как «I/O не работает» — а на самом деле просто ему не дают слот.

05 При оптимизации тяжёлых задач не всегда очевидно, на какой тип задачи разбивать: `setTimeout(0)` — простой, `MessageChannel` — без 4мс клампа, `scheduler.postTask` — с явным приоритетом.

КОД

// Микро-старвейшен

```
function loop() { Promise.resolve().then(loop); }
loop();
// макрозадачи никогда не дойдут
// рендер никогда не случится
```

// Чанк через `setTimeout`

```
function chunk(items, work) {
  if (!items.length) return;
  const slice = items.splice(0, 100);
  slice.forEach(work);
  setTimeout(() => chunk(items, work), 0);
}
```

// Чанк через `scheduler.postTask`

```
function chunk(items, work) {
  if (!items.length) return;
  const slice = items.splice(0, 100);
  slice.forEach(work);
  if ('scheduler' in window) {
    scheduler.postTask(
      () => chunk(items, work),
      { priority: 'background' }
    );
  } else {
    setTimeout(() => chunk(items, work), 0);
  }
}
```

// Опасный `nextTick` в Node

```
function loop() { process.nextTick(loop); }
loop();
// все фазы event loop встанут
```

МОГУТ ДОСПРОСИТЬ

01 Что такое `scheduler.postTask` и зачем его добавили в браузер?

02 Чем `nextTick` опаснее `queueMicrotask` в Node?

03 Как поймать микро-старвейшен в Performance-вкладке DevTools?

<https://learn.javascript.ru/microtask-queue>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Своими словами объяснить event loop, без зубрёжки чужих формулировок (Q1)

- ☐ Развести task и microtask на конкретных API (Q2)

- ☐ Прогнать пазл порядка sync / setTimeout / Promise.then / queueMicrotask / rAF (Q3)

- ☐ Сказать, когда браузер успеваеt paint и почему долгий for его блокирует (Q4)

- ☐ Объяснить, почему бесконечная микроочередь блокирует UI, а конечная – нет (Q5)

- ☐ Сравнить all / allSettled / race / any одной фразой каждое (Q6)

- ☐ Описать race condition и две стратегии защиты на пальцах (Q9, Q10)

- ☐ Написать retry с exponential backoff на доске (Q7)

- ☐ Отменить fetch через AbortController и комбинировать сигналы (Q8)

- ☐ Реализовать pool с лимитом параллелизма и объяснить, почему он лучше chunk-разбиения (Q13)

- ☐ Развести debounce и throttle на сценариях (Q14, Q15)

- ☐ Сказать, когда rAF, а когда rIC (Q16, Q17)

- ☐ Объяснить starvation и одну стратегию лечения (Q18)
